ParCFD2025
36th International Conference on Parallel Computational Fluid Dynamics
Nov. 24-26, 2025, Mérida,Yucatán, Mexico

Evaluating the Computational Performance and Accuracy of a Coupled CFD Solver-ML Workflow

Tom Hilgers[a,*], Fabian Orland[a], Fabian Hübenthal[b], Rakesh Sarma[c], Andreas Lintermann[c] and Christian Terboven[a]

[a]*RWTH Aachen University, Chair for High-Performance Computing Seffenter Weg 23, 52074 Aachen, Germany*
[b]*RWTH Aachen University, Institute of Aerodynamics and Chair of Fluid Mechanics Wüllnerstraße 5a, 52062 Aachen, Germany*
[c]*Forschungszentrum Jülich GmbH, Jülich Supercomputing Centre Wilhelm-Johnen-Straße, 52425 Jülich, Germany*

ARTICLE INFO[†]

Keywords:
Transformers;
Turbulent Boundary Layers;
Computational Fluid Dynamics / Artificial Intelligence Coupling;
Performance Evaluation;
PhyDLL

ABSTRACT

Recent advances in Machine Learning (ML) architectures offer new opportunities to accelerate large-scale Computational Fluid Dynamics (CFD) simulations, particularly for Turbulent Boundary Layer (TBL) flows that require substantial hardware resources to sufficiently resolve the complex flow features. Building on prior work that proposed coupling a Transformer model for TBL problems with the multi-physics solver m-AIA, we implement the approach with two interchangeable coupling libraries and investigate its potential and limitations in reducing computational costs and preserving accuracy. By periodically replacing solver steps with GPU-based Transformer predictions, we find a single node speedup of 11% and demonstrate non-stagnating strong scalability on multiple nodes with 80% parallel efficiency. Accuracy assessments show <10% numerical error for the coupled workflow, while a standalone evaluation of the Transformer reveals substantially lower physical accuracy, with up to 70% error. Our findings demonstrate the scalability of the coupling implementation while highlighting the runtime and accuracy limitations of the investigated ML approach.

1. Introduction

Accurate and efficient turbulent flow simulations remain a central challenge in Computational Fluid Dynamics (CFD). While high-fidelity solvers can capture intricate and fine flow field structures, they typically require prohibitive computational resources, especially in large-scale time-marched scenarios. Advances in Machine Learning (ML) present an opportunity to reduce these costs by using such data-driven models to approximate the flow evolution for intermediate time steps, while retaining the numerical solver for the rest of the time steps [1]. In this context, Transformer-based architectures, in particular, have gained traction for their ability to capture long-term temporal dependencies in complex flow fields [2].

However, integrating ML models into existing High Performance Computing (HPC)-optimized CFD workflows is not trivial and raises concerns about resource allocation, scalability and long-term robustness of simulation results. It is therefore crucial to systematically evaluate the trade-off between accelerating simulations and maintaining adherence to the underlying physics. In addition, practical considerations, such as the allocation of costly hardware resources, further underscore the need for careful and quantitative evaluations.

Previous works developed a Transformer model for a Turbulent Boundary Layer (TBL) problem to evaluate speedups [3] relative to traditional time-stepping with the highly parallel, multi-physics solver m-AIA [4]. They also proposed a Physics Deep Learning coupLer (PhyDLL)-based [5] coupling framework [6] to unify the Transformer and solver in a single workflow, in line with the ML–assisted simulation paradigm outlined by von Rueden et al. [7]. Building on these works, we implement this workflow using two interchangeable coupling libraries and assess whether it can accelerate TBL simulations without incurring significant loss of accuracy.

In Section 2, the implemented coupling scheme is presented and a detailed evaluation of its resulting workflow in terms of computational performance and simulation fidelity is given in Section 3. Additionally, we investigate the model itself in Section 4 using physics-based metrics and discuss underlying architectural weaknesses.

*Corresponding author
tom.hilgers@rwth-aachen.de (T. Hilgers); orland@itc.rwth-aachen.de (F. Orland); f.huebenthal@aia.rwth-aachen.de (F. Hübenthal); r.sarma@fz-juelich.de (R. Sarma); a.lintermann@fz-juelich.de (A. Lintermann); terboven@itc.rwth-aachen.de (C. Terboven)
ORCID(s): 0000-0002-7501-3936 (T. Hilgers); 0000-0002-8681-2661 (F. Orland); 0009-0000-7159-8220 (F. Hübenthal); 0000-0002-7069-4082 (R. Sarma); 0000-0003-3321-6599 (A. Lintermann); 0000-0003-2284-2957 (C. Terboven)

2. Transformer - CFD Coupling for TBL Problem

In this work, we implement and evaluate the coupling framework proposed by Sarma et al. [6] on a flat-plate TBL problem. For the specific TBL formulation and Transformer training details, we refer to our previous works [3, 8], as no changes have been made to these. Additionally, two extensions to the original workflow have been made.

2.1. Extension 1: Time Marching Scheme

During training, the Transformer predictions agreed with the m-AIA reference within 5% error [3]. However, using the model in every time step would accumulate these small errors over time and destabilize the simulation. To mitigate this, we extend the existing coupling workflow by introducing a flexible time marching scheme that controls the frequency of the Transformer prediction according to a parameter N_{cfd}.

Under this scheme, after each Transformer prediction, the m-AIA solver will perform N_{cfd} regular physical simulation steps before invoking the next prediction. This allows us to investigate whether m-AIA correctly incorporates the fields predicted by the Transformer and how many solver steps remain necessary to maintain both a stable and accurate solution. A simplified overview of the scheme is shown in Figure 1.

Consequently, each time step now falls into one of three categories: 1. regular solver step (black), 2. coupling step (orange), or 3. Transformer step (red). Suppose that, at time step t, a Transformer prediction is due. For an input sequence of m velocity fields, $m-1$ preceding time steps must be coupling steps. In these, the velocity fields are communicated to the model, but no inference of the Transformer is performed. Actual inference occurs only in a Transformer step, where the same communication is followed by the model inference. The model was trained using a distance of $n = 24$ solver time steps between sequence elements, but we found a distance of one to be sufficient.

After the model has been inferred, the simulation time inside m-AIA needs to be advanced by 24 time steps to match the intervals used during training [3]. The next $N_{\text{cfd}} - mn$ time steps are regular m-AIA simulation steps, neither communicating data nor inferring the Transformer model. Then m coupling steps, distanced by n regular steps, follow in preparation for the next Transformer step at time step $t + 24 + N_{\text{cfd}}$ The whole process repeats itself until the simulation is complete.

2.2. Extension 2: Modular Coupling Framework

Following Orland et al. [9], we encapsulate the time marching scheme and coupling workflow in a single `ml_step` procedure with three sub-steps: `preprocess_input`, `inference`, and `postprocess_output`. The preprocessing partitions the simulation domain into cubic subdomains, while the inference step is calling a coupling library to perform the inference, and postprocessing reassembles the domain. This modular design allows us to implement scalable, parallel, GPU-accelerated Transformer inference interchangeably via either PhyDLL [5] or the AIxeleratorService [9].

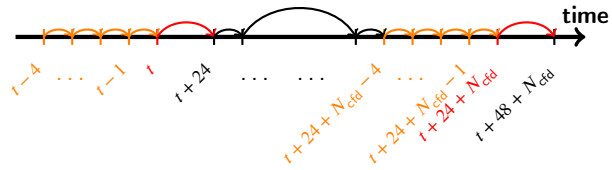

Figure 1: m-AIA + Transformer time stepping scheme with regular solver steps in black, coupling steps in orange, and inference steps in red. For clarity, the input sequence is set to a length of $m = 5$ consecutive $(n = 1)$ elements.

3. Coupled Workflow Evaluation

We now quantitatively examine the performance of the coupled workflow, described in Section 2. We employ the Transformer model to accelerate the simulation while aiming to achieve a high level of accuracy. Our objective is to evaluate whether this premise holds, identify the optimal coupling configuration, and assess the resulting simulation accuracy.

We first assess the accuracy of the coupled simulation using a version of the TBL problem, where the turbulence is fully formed but the plate is not actuated, over 100,000 simulation steps. This is performed on a small grid of 172×65×15 cells, minimizing resource usage and facilitating a faster evaluation. We track two main outputs:

1. Drag forces: A force value (N/m^2) recorded in each time step consisting of viscous and pressure forces on the plate, central to calculating drag reduction for the actuated plate [10].
2. Flow fields: 3D velocity fields given in cartesian components, saved every 1000 time steps, which show how the solution evolves over time, reflecting the Transformer training [3].

We quantify simulation accuracy by computing two error metrics on the streamwise 3D velocity field, as it predominantly influences drag. Let the time index t denote the current timestep, and let $V_{\text{ref}}^{(t)}$ and $V_{\text{ml}}^{(t)}$ be the reference and coupled fields, respectively. The first metric is the relative euclidian norm error, $E_{L2,rel}^{(t)} = \frac{\|V_{\text{ref}}^{(t)} - V_{\text{ml}}^{(t)}\|_2}{\|V_{\text{ref}}^{(t)}\|_2}$, and the second metric is the mean absolute percentage error (MAPE).

To integrate the Transformer into the m-AIA solver effectively, we must find the optimal ratio of classical simulation steps to machine learning predictions. We therefore examine the effect of different values, that are 5, 10, 25, 50, 75, and

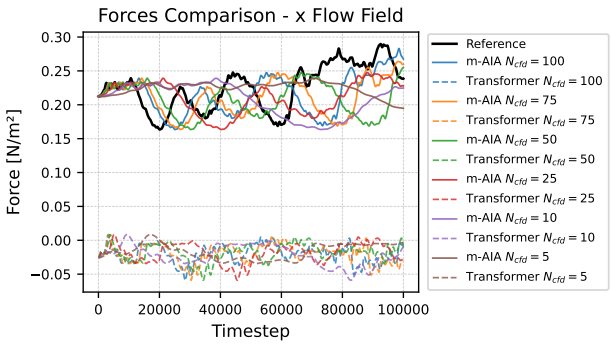

Figure 2: Comparison of forces in the streamwise flow field between m-AIA time steps and Transformer predictions for different values of N_{cfd}.

100 of the parameter N_{cfd}, on overall simulation accuracy, comparing against uncoupled reference simulations. From these tests, we can identify which ratio delivers the best balance of accuracy and runtime performance. To mitigate the effect of outliers, we perform five simulation runs for each configuration, averaging the metrics per time-step over the runs. We use $m = 5$ with consecutive ($n = 1$) sequence elements in this section.

Our second evaluation assesses whether the modular coupling using PhyDLL or the AIxeleratorService delivers accelerated as well as scalable simulations. We use GPU nodes of the CLAIX-2023 High Performance Computing (HPC) cluster, each consisting of four Nvidia H100 GPUs and two 48-core Intel Xeon 8468 CPUs, and use $N_{\text{cfd}} = 75$ for Transformer predictions. Due to the Multiple Program Multiple Data (MPMD) approach of PhyDLL the user has to either allocate additional cores, for example on another node, for the Deep Learning (DL) processes or divide the available cores between the CFD solver and the DL processes on the same node. First, we demonstrate the performance impact of the latter scenario by allocating different numbers of CPU cores per GPU, and compare against an uncoupled CPU-only reference simulation using all 96 CPU cores. The investigated values range from 1 to 12 cores per GPU, resulting in 92 to 48 cores for the m-AIA solver.

Next, we perform multi-node, strong scaling experiments on a large $732 \times 130 \times 250$ grid for 3,000 simulation steps, with $N_{\text{cfd}} = 5$, on 1 to 8 nodes, using the AIxeleratorService. Unlike PhyDLL, its Single Program Multiple Data (SPMD) approach requires no additional CPU cores, thereby resolving a key compromise inherent to PhyDLL. Moreover, previous work has demonstrated superior scalability for the AIxeleratorService [9].

3.1. Physical Accuracy

In this section, we present the results of our first evaluation, which addresses the accuracy of the coupled workflow.

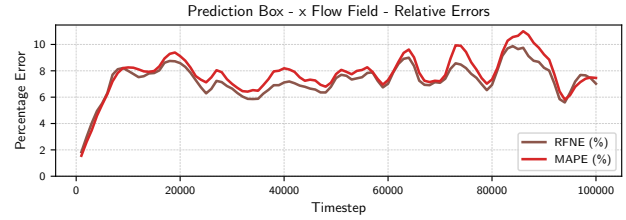

Figure 3: Error metrics of the coupled simulation, with $N_{\text{cfd}} = 75$, in the region of interest (prediction box).

Figure 2 shows the forces acting on the plate in the streamwise direction. Each line color corresponds to one value of N_{cfd}, with the uncoupled reference in black. For clarity, we plot simulation and coupling step values with a solid line, and Transformer step values with a dashed line.

Notably, the forces of the Transformer predictions are mostly negative, whereas the classical simulation never produces negative drag. This implies a nonphysical acceleration of the plate. We found the cause in the predicted velocity at the plate, where some values are negative and thus yield negative gradients. Enforcing a no-slip boundary constraint in the Transformer might improve this behavior.

Despite this noticeable discrepancy, the regular solver steps largely compensate for it, keeping results in the same general region as the reference simulation. We observe that all cases follow a very similar general path, but each decrease in N_{cfd} shifts and stretches the path further to the right, i.e. into the future. This means that the simulation takes more counted time steps to follow the reference path, but internally takes the same amount of regular simulation steps, since some are skipped by the transformer, a behavior that we are still investigating.

Regarding the second output, Figure 3 presents only $N_{\text{cfd}} = 75$ for conciseness, since we also examine it further in Section 3.2. Both errors begin low, rise quickly, then stabilize between 6% and 10% after 10,000 time steps, which reflects the significant force deviations. As context, drag reduction via actuation reported by Albers et al. [10] lies between 10% and 27%. Therefore, the coupled workflow is not yet meeting the accuracy requirements for evaluating actuation parameters. We next address the runtime performance of the coupling.

3.2. Computational Performance

In this section, we present the results of the runtime evaluation. For the single node performance we chose $N_{\text{cfd}} = 75$, as this utilizes the Transformer model more frequently than the $N_{\text{cfd}} = 100$ configuration, thereby promising a greater overall performance gain with similar accuracy. For the strong scaling evaluation however, $N_{\text{cfd}} = 5$ lets us investigate the scaling behavior of the `ml_step` at lower overall computational cost.

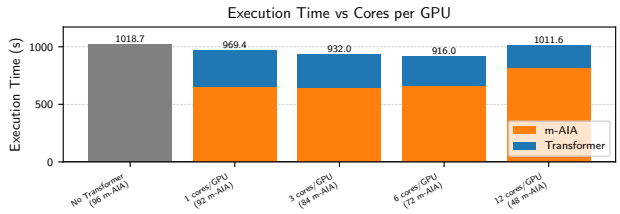

Figure 4: Absolute execution times of m-AIA with and without the Transformer coupling using PhyDLL in different configurations on a single node with $N_{\text{cfd}} = 75$.

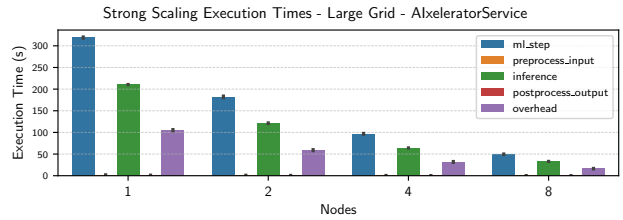

Figure 5: Strong scalability of the coupling workflow using the AIxeleratorService on up to 8 GPU nodes.

3.2.1. Single Node Performance using PhyDLL

Figure 4 shows the performance impact of dedicating different numbers of cores to the m-AIA simulation and the DL processes. The blue part of the bars highlights the cost of inferring the Transformer, including communication and computation. All Transformer-coupled configurations achieve some improvement, though the magnitude varies. The best performance occurs with six cores per GPU and 72 solver cores. The runtime initially improves with more cores per GPU, but allocating 12 cores degrades overall performance again. Thus, the optimal trade-off is at six cores per GPU.

In the best case, we observe an absolute improvement of about 102 seconds, or 11% speedup. This speedup is lower than expected when taking into account the high total peak performance of the four GPUs as well as the weak baseline [11] we compare against. We suspect that the GPU support for the solver, that is currently under development, could potentially outperform the Transformer-based approach.

3.2.2. Strong Scalability using AIxeleratorService

Figure 5 presents the strong scaling results up to 8 GPU nodes. The blue, leftmost bar shows the total runtime for the whole `ml_step`, while the purple overhead is computed as $t_{\texttt{overhead}} = t_{\texttt{ml_step}} - t_{\texttt{preprocess}} - t_{\texttt{inference}} - t_{\texttt{postprocess}}$ and thus includes time for MPI communication, batched tensor creation and device data transfers inside the AIxeleratorService library. The dominant sub-steps of the coupling workflow scale well with increasing resources. On 8 nodes, the parallel efficiency is 80% for the whole `ml_step`, 80% for the `inference`, and 81% for the overhead. Although the overhead scales well, it accounts for 32% of the runtime on 8 nodes. Deeper analysis revealed that device data transfers are responsible for 69% of the overhead with MPI communication accounting for the remaining 31%. We also found that increasing the number of ranks slightly increases the total workload due to suboptimal preprocessing caused by changing the domain decomposition. On 8 nodes, this causes a 7% workload increase, which limits the ideal parallel efficiency to 93%. Reaching 80% parallel efficiency therefore underscores the scalability of the AIxeleratorService and our coupling approach.

4. Standalone Model Evaluation

Section 3.1 and in particular Figure 2 demonstrate a clear mismatch between the model predictions and the simulation, whereas Sarma et al. [3] report very low errors. To identify the reason behind this discrepancy, we now evaluate the model directly without the solver coupling. We compute the L2_{rel} error for three quantities: the streamwise velocity component, the vorticity, and the drag force from Section 3.1. The latter two are physics-based and therefore fitting for this TBL simulation. As in the model training, solver outputs serve as ground truth.

To obtain a comprehensive and unbiased assessment, we perform this evaluation on several grids with varying wave parameters (see Appendix A for details). Two model uses are considered, teacher forcing where only ground truths serve as model inputs, mirroring the coupled use, and autoregressive rollout where previous predictions are reused as input. As in training, the model receives $m = 3$ inputs, each separated by $n = 24$ solver time steps (depending on the grid resolution) and a forecast window of the same value. We make 12 predictions in both modes, calculating the mean error for teacher forcing as they differ only marginally.

4.1. Results

The error values for teacher forcing in Figure 6 explain the optimistic results of the previous work. The direct error on the streamwise velocity is low at less than 2.5% in the worst case. Although we see noticeable relative differences between the cases, they are small in absolute terms. In stark contrast, the physics-based metrics reveal a substantially worse performance. The vorticity error ranges from 12% to 24%, with a similar relative distribution to the direct error. The wall-resolved drag error is significantly higher at around 70% for all cases. The predicted force magnitudes are mostly negative and are caused by negative velocities in the wall-adjacent layer, which resembles the behavior seen in Figure 2.

Figure 7 shows that when the model is used autoregressively, the errors accumulate and grow linearly with each consecutive prediction for nearly all cases and metrics. Some cases exhibit a slower growth in the velocity error, but the overall trend persists. Therefore, no long term stability is

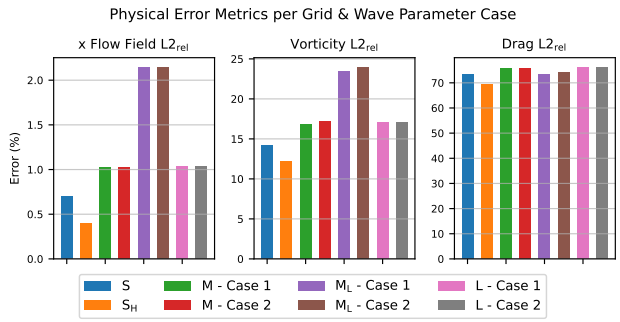

Figure 6: Mean error metrics of the standalone model evaluation for teacher forcing.

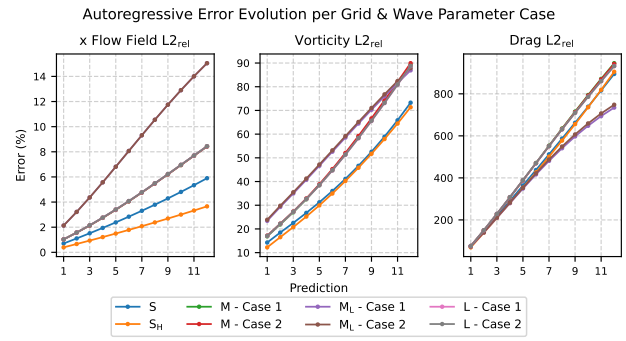

Figure 7: Error metrics per prediction of the standalone model evaluation for autoregressive prediction.

given and it follows that predictions should not be made autoregressively in order to minimize error.

These findings show that, in its current form, the model cannot generate physically accurate flow fields. The low direct errors alone are misleading, instead physics-based, task-specific metrics are essential for a meaningful evaluation.

A limitation of the model lies in the domain decomposition into cubic subdomains. Each temporal sequence of cubes is processed in isolation, depriving the model of information about neighboring cells. This lack of larger spatial context is especially detrimental in a turbulent flow, where turbulent structures can span multiple cubes. Because the loss is computed per cube, the training cannot compensate for these missing interactions and even introduces inaccuracies at the edges. Introducing a context halo, which is standard practice in conventional domain decomposing CFD solvers, could mitigate this deficiency.

Other architectural limitations also constrain the performance. In our experiments, the encoder produces the same output representation for all subdomains, meaning that it does not provide useful information to the decoder. Furthermore, a longer input sequence does not provide any benefit, with a sequence length of $m = 1$ reaching similar errors as $m = 3$. Therefore, the temporal attention does not benefit from adding more context. Given the characteristics of the TBL problem, spatial or spatio-temporal attention may be more effective. Neural operator-based approaches present an alternative direction, but their efficacy remains debated [11]. For a more detailed discussion of the Transformer, including an ablation study, we refer to [12].

5. Conclusions

In this paper, we implemented and evaluated a previously proposed CFD–ML coupling for both accuracy and runtime. Specifically, a Transformer model was coupled to the m-AIA solver and invoked in regular intervals. We observed negative drag values in the model's wall-adjacent predictions and values of 75 and 100 for N_{cfd} showed a 6–10% numerical error, relative to the uncoupled reference simulation.

A standalone evaluation reveals that, while the model achieves low errors on the raw velocity values, it performs significantly worse when assessed with physics-based metrics, reaching errors of up to 70%. Autoregressive rollout leads to linearly accumulating errors, discouraging its use in practice. Limitations are found in the decomposition into isolated cubic subdomains, which deprives the model of spatial context, the encoder representations, and also the temporal attention mechanism, which does not benefit from larger contexts. These findings show the need for application specific metrics in evaluating ML models and architectural improvements of the used Transformer model.

Along with the high error rates, replacing solver steps with GPU-based ML predictions yields a runtime improvement of up to 11% on a single compute node with PhyDLL. Nevertheless, our coupling implementation achieves non-stagnating strong scalability with the AIxeleratorService, reaching a parallel efficiency of 80% on 8 nodes. In summary, in its current form, the Transformer model fails to deliver both the desired significant acceleration of the CFD simulation and the required accuracy.

Acknowledgements

The authors gratefully acknowledge the computing time provided to them on the high performance computer CLAIX-2023 at the NHR Center RWTH Aachen University. This is funded by the German Federal Ministry of Research, Technology and Space and the state government of North Rhine-Westphalia. The computations for this research were performed using computing resources under projects thes1961, rwth1859 and p0025821.

References

[1] K. Duraisamy, G. Iaccarino, H. Xiao, Turbulence Modeling in the Age of Data, Annual Review of Fluid Mechanics 51 (2019) 357–377. doi:10.1146/annurev-fluid-010518-040547.

[2] M. Z. Yousif, M. Zhang, L. Yu, R. Vinuesa, H. Lim, A transformer-based synthetic-inflow generator for spatially developing turbulent boundary layers, Journal of Fluid Mechanics 957 (2023) A6. doi:10.1017/jfm.2022.1088.

[3] R. Sarma, F. Hübenthal, E. Inanc, A. Lintermann, Prediction of Turbulent Boundary Layer Flow Dynamics with Transformers, Mathematics 12 (2024) 2998. doi:10.3390/math12192998.

[4] A. Lintermann, M. Meinke, W. Schröder, Zonal Flow Solver (ZFS): a highly efficient multi-physics simulation framework, International Journal of Computational Fluid Dynamics 34 (2020) 458–485. doi:10.1080/10618562.2020.1742328.

[5] A. Serhani, V. Xing, D. Dupuy, C. Lapeyre, G. Staffelbach, Graph and convolutional neural network coupling with a high-performance large-eddy simulation solver, Computers & Fluids 278 (2024) 106306. doi:10.1016/j.compfluid.2024.106306.

[6] R. Sarma, F. Hübenthal, F. Orland, C. Terboven, A. Lintermann, Predicting Turbulent Boundary Layer Flows using Transformers coupled to the Multi-Physics Simulation Tool M-AIA, in: 35th Parallel CFD International Conference 2024, FZJ-2025-02459, Jülich Supercomputing Center, 2024. doi:10.34734/FZJ-2025-02459.

[7] L. von Rueden, S. Mayer, R. Sifa, C. Bauckhage, J. Garcke, Combining Machine Learning and Simulation to a Hybrid Modelling Approach: Current and Future Directions, in: M. R. Berthold, A. Feelders, G. Krempl (Eds.), Advances in Intelligent Data Analysis XVIII, Springer International Publishing, 2020, pp. 548–560. doi:10.1007/978-3-030-44584-3_43.

[8] R. Sarma, E. Inanc, M. Aach, A. Lintermann, Parallel and scalable AI in HPC systems for CFD applications and beyond, Frontiers in High Performance Computing 2 (2024). doi:10.3389/fhpcp.2024.1444337.

[9] F. Orland, L. Nista, N. Kocher, J. Vanvinckenroye, H. Pitsch, C. Terboven, Efficient and Scalable AIxeleration of Reactive CFD Solvers Coupled with Deep Learning Inference on Heterogeneous Architectures, in: Proceedings of the 2025 International Conference on High Performance Computing in Asia-Pacific Region Workshops, HPC Asia '25 Workshops, Association for Computing Machinery, 2025, pp. 45–57. doi:10.1145/3703001.3724386.

[10] M. Albers, P. S. Meysonnat, D. Fernex, R. Semaan, B. R. Noack, W. Schröder, Drag Reduction and Energy Saving by Spanwise Traveling Transversal Surface Waves for Flat Plate Flow, Flow, Turbulence and Combustion 105 (2020) 125–157. doi:10.1007/s10494-020-00110-8.

[11] N. McGreivy, A. Hakim, Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations, Nature Machine Intelligence 6 (2024) 1256–1269. doi:10.1038/s42256-024-00897-5.

[12] T. Hilgers, Coupling Turbulent Boundary Layer Flow Simulation with a Transformer, Master's thesis, RWTH Aachen University, 2025. doi:10.18154/RWTH-2025-10668.

A. Standalone Model Evaluation Details

The evaluation in Section 4 was performed on the cases shown in Table 1. The physical resolution of the M_L cases was 60% lower in the x direction, and 50% lower for both the general z- and wall-near y-directions. For the S_H case the increases in resolution were 20%, 50%, and around 43%. A change in resolution directly affects the physical time covered by a solver step and therefore also the time covered by a prediction, affecting parameter n. All shown error values are exclusively calculated in the region of interest, as specified in previous works [3, 10].

Case	Grid dimensions	Wave λ^+, T^+, A^+	n
L - Case 1	$732 \times 130 \times 250$	500, 20, 30	24
L - Case 2	$732 \times 130 \times 250$	500, 40 ,21	24
M - Case 1	$732 \times 130 \times 125$	250, 30, 30	24
M - Case 2	$732 \times 130 \times 125$	500, 50, 40	24
M_L - Case 1	$439 \times 66 \times 62$	250, 30 ,30	40
M_L - Case 2	$439 \times 66 \times 62$	500, 50 ,40	40
S	$732 \times 130 \times 31$	None	24
S_H	$878 \times 233 \times 62$	None	20

Table 1
Grid dimensions (x,y,z) and wave parameters (wavelength, period, amplitude) in inner units for each case of the standalone model evaluation. The subscripts $_L$ and $_H$ signal a lower or higher resolution version of a grid with the same physical size.